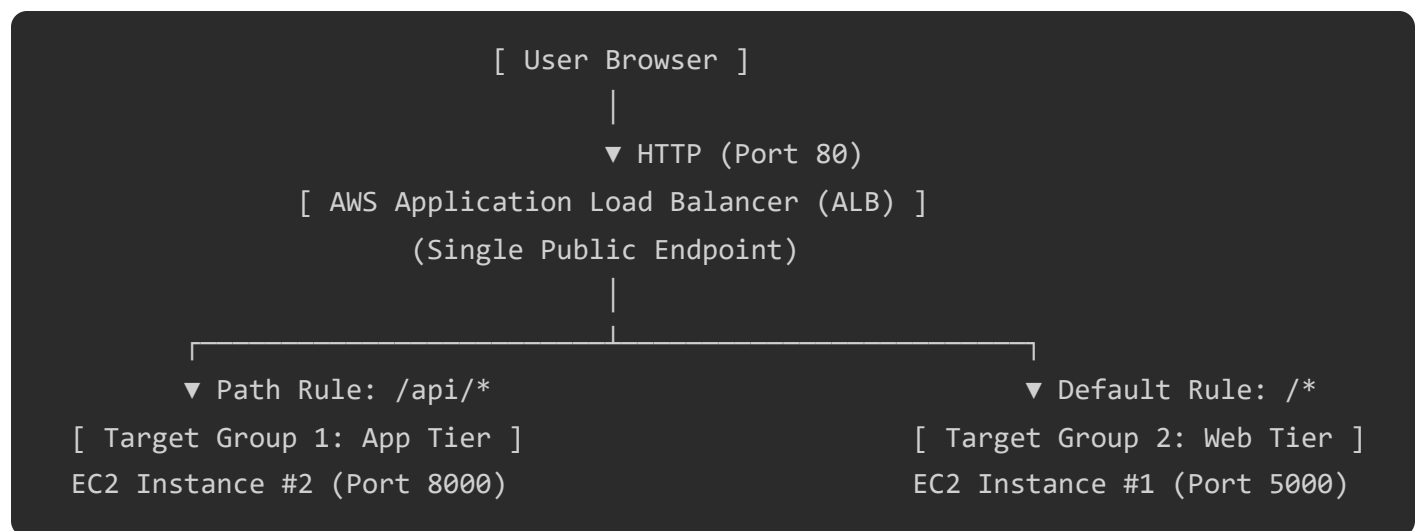


AWS Application Load Balancer (ALB)

Deployment Guide

This guide details how to add an **AWS Application Load Balancer (ALB)** in front of your Web Tier and App Tier using **Path-Based Routing** (`/api/*` vs `/*`).

Architecture Overview



Why ALB Path-Based Routing is Enterprise Standard:

1. **Single Entry Point:** Your users only connect to one URL (the ALB DNS name or custom domain).
 2. **Relative API URLs:** Frontend `config.js` uses `window.API_BASE = "/api"`. No hardcoded IP addresses needed!
 3. **High Availability & Auto-Healing:** ALB automatically monitors target health and routes around failed instances.
-

Step 1: Create Target Group 1 (App Tier REST API)

1. Go to **AWS Console** → **EC2** → **Target Groups** → **Create target group**.
2. **Basic Configuration:**
 - **Target type:** Instances

- **Target group name:** avashya-app-tg
 - **Protocol:** HTTP
 - **Port:** 8000
 - **VPC:** Default VPC
3. **Health checks:**
- **Health check path:** /api/users
4. Click **Next**.
5. Select **App EC2 Instance #2** (Avashya-App-Tier-EC2), click **Include as pending below**, and click **Create target group**.
-

Step 2: Create Target Group 2 (Web Tier Frontend)

1. Go to **AWS Console** → **EC2** → **Target Groups** → **Create target group**.
 2. **Basic Configuration:**
 - **Target type:** Instances
 - **Target group name:** avashya-web-tg
 - **Protocol:** HTTP
 - **Port:** 5000
 - **VPC:** Default VPC
 3. **Health checks:**
 - **Health check path:** /index.html
 4. Click **Next**.
 5. Select **Web EC2 Instance #1** (Avashya-Web-Tier-EC2), click **Include as pending below**, and click **Create target group**.
-

Step 3: Create the Application Load Balancer (ALB)

1. Go to **AWS Console** → **EC2** → **Load Balancers** → **Create load balancer**.
2. Select **Application Load Balancer (ALB)** → Click **Create**.
3. **Basic Configuration:**
 - **Load balancer name:** avashya-alb
 - **Scheme:** Internet-facing
 - **IP address type:** IPv4
4. **Network mapping:**
 - **VPC:** Select Default VPC
 - **Mappings:** Select at least 2 Availability Zones (e.g. ap-south-1a, ap-south-1b).

5. Security groups:

- Select or create a Security Group `avashya-alb-sg` with **Inbound Rule: HTTP Port 80 from 0.0.0.0/0**.

6. Listeners and routing:

- **Protocol:** HTTP | **Port:** 80
- **Default action:** Forward to `avashya-web-tg`.

7. Click **Create load balancer**.

Step 4: Configure Path-Based Routing Rule (`/api/*`)

1. Go to **EC2 Console** → **Load Balancers** → **Select** `avashya-alb`.
2. Click the **Listeners** tab → Click **HTTP:80**.
3. Click **Add rule** → Name it `Route-API-Traffic`.
4. **Conditions:** Add condition → **Path** → Value: `/api/*`.
5. **Actions:** Forward to `avashya-app-tg`.
6. Click **Save / Create**.

Step 5: Update Frontend `config.js` for ALB

With ALB path routing in place, frontend `config.js` can use relative `/api` paths!

Update `frontend/config.js` on Web EC2 Instance #1:

```
// Avashya Drop Web Tier Config (ALB Path-Based Routing)
window.API_BASE = "/api";
console.log('[Avashya Drop Web Tier] ALB Connected API Endpoint:', window.API_BASE);
```



Step 6: Test Your ALB Deployment

1. Copy the **DNS name** of your ALB from the AWS Console (e.g. `avashya-alb-123456789.ap-south-1.elb.amazonaws.com`).
2. Open your web browser and visit: 🖱️ `http://<ALB_DNS_NAME>`

Your browser will load the Web Tier, login via `/api/auth/login`, upload files to S3, and query Amazon RDS—all through a single Load Balancer endpoint!

